

CSE 101 - Local Development with Docker

This guide helps you run your C programs in a **Linux environment** that matches the one used for grading.

Why use Docker?

Your code will be graded in a Linux environment with:

- **Ubuntu 22.04**
- **gcc (C compiler)**
- **make**
- **valgrind** (for memory checks)

Using Docker lets you run your code locally under the **same environment**, so you avoid “works on my machine” issues.

If you choose not to use Docker, you are responsible for making sure your environment matches this setup.

1. Install Docker

Install Docker on your machine:

- Official guide: <https://docs.docker.com/get-docker/>
- Windows / macOS: install **Docker Desktop**
- Linux: install **Docker Engine**

Verify installation:

```
docker --version
docker run --rm hello-world
```

2. Set up your project

Place the provided Dockerfile inside your assignment folder.

Example:

```
hw0/  
  main.c  
  Makefile  
  Dockerfile
```

3. Build the environment (one-time)

From inside your assignment folder:

```
docker build -t cse101-build .
```

This creates a local Linux environment with the required tools.

4. Compile your code

Run make inside the container:

macOS / Linux

```
docker run --rm -v "${PWD}:/workspace" -w /workspace cse101-build make
```

Windows (PowerShell)

```
docker run --rm -v "${PWD}:/workspace" -w /workspace cse101-build make
```

5. Run your program

After compiling:

```
docker run --rm -v "${PWD}:/workspace" -w /workspace cse101-build ./main
```

Your program name depends on your Makefile. Replace `./main` with your executable name if different.

6. (Optional) Check memory with valgrind

```
docker run --rm -v "${PWD}:/workspace" \  
-w /workspace cse101-build valgrind --leak-check=full ./main
```